PEMROGRAMAN BERORIENTASI OBJEK 2026

Proyek Terintegrasi

Merakit KedaiKita menjadi aplikasi kasir sungguhan: berlapis, menyimpan data ke *file*, dan teruji otomatis

PERTEMUAN

15

dari 16

Program Studi Informatika · Universitas Mulawarman
Jalur praktik: Python dan Java

Tujuan Pembelajaran

Setelah pertemuan ini, kamu mampu:

1. Menjelaskan alur membangun aplikasi: **kebutuhan → rancangan UML → implementasi → pengujian**.
2. Menjelaskan **arsitektur berlapis** (tampilan, layanan, model, penyimpanan) beserta manfaatnya.
3. Mengelompokkan kelas ke dalam **modul** dan ***package***.
4. Menyimpan dan memuat data objek dengan *file* **CSV**, termasuk menolak isi *file* yang rusak dengan pesan yang jelas.
5. Membaca masukan pengguna dengan **aman**: validasi, tanya ulang, dan tangani akhir *input*.
6. Menulis ***unit test*** sederhana dan menjelaskan mengapa pengujian otomatis penting.

Mengingat Kembali



Dari pertemuan sebelumnya

- Pertemuan 14: KedaiKita mendapat *Singleton*, *Factory*, *Strategy*, dan *Observer*.
- Pertemuan 13: tanggung jawab dipisah (SRP) dan `PenyimpananPesanan` disuntikkan ke `Kedai` (DIP).
- Pertemuan 11: kita mengintip `with` (Python) dan *try-with-resources* (Java) untuk *file*.
- Masalahnya, data KedaiKita versi Pertemuan 14 masih ditulis di kode dan **hilang** saat program ditutup.

Agenda Hari Ini

1. Dari latihan ke aplikasi nyata
2. Arsitektur berlapis dan *package*
3. Menyimpan data ke *file*
4. *Input* aman dan *unit test*
5. Studi kasus KedaiKita: aplikasi kasir utuh

BAGIAN 1

1

Dari Latihan ke Aplikasi Nyata

Program latihan cukup membuktikan konsep; aplikasi nyata dipakai orang setiap hari.

Program Latihan dan Aplikasi Nyata

Program latihan	Aplikasi nyata
Data ditulis langsung di kode	Data dibaca dari *file*, dan perubahan disimpan kembali
Dijalankan sekali lalu selesai	Dipakai berulang kali; data hari ini masih ada besok
Masukan sudah “pasti benar”	Pengguna bisa salah ketik kapan saja
Diperiksa dengan mata: “kelihatannya benar”	Diperiksa otomatis oleh *unit test* setiap kali kode berubah
Satu-dua *file*	Puluhan *file* yang perlu ditata

Alur Membangun Aplikasi

1. **Kebutuhan**: apa yang harus bisa dilakukan aplikasi.
2. **Rancangan**: diagram kelas UML.
3. **Implementasi**: sedikit demi sedikit, setiap potongan langsung dicoba.
4. **Pengujian**: *unit test* dan uji coba. Jika ada yang kurang atau salah, kembali ke langkah 1.

Temuan dari pengujian sering memunculkan kebutuhan baru, jadi alurnya berputar.

Kebutuhan KedaiKita Hari Ini

Fungsional: bisa melakukan apa

- **F1** Daftar menu dibaca dari *file*, bukan ditulis di kode.
- **F2** Kasir membuat pesanan, memilih diskon, dan membayar lewat menu teks.
- **F3** Stok terbaru disimpan saat aplikasi ditutup dan dimuat lagi saat dibuka.
- **F4** Setiap pesanan LUNAS tercatat di *file* riwayat transaksi.
- **F5** Ada laporan penjualan dan riwayat transaksi.

Nonfungsional: seberapa baik

- **N1** Salah ketik tidak membuat aplikasi berhenti mendadak.
- **N2** *File* data yang rusak dilaporkan dengan pesan jelas, lengkap dengan nomor barisnya.
- **N3** Perhitungan penting diperiksa otomatis oleh *unit test*.
- **N4** Ada mode demo yang berjalan tanpa diketik, dengan hasil yang selalu sama.

BAGIAN 2

2

Arsitektur Berlapis dan *Package*

Menata tugas ke dalam lapisan, lalu menata *file* ke dalam *package*.

Restoran yang Ramai di Dekat Kampus

Analogi

- **Pelayan** mencatat pesanan dan mengantar makanan, tetapi tidak memasak. **Kepala dapur** mengatur urutan masakan.
- **Juru masak** mengolah bahan sesuai resep. **Gudang dan buku catatan** menyimpan bahan dan riwayat penjualan.
- Pelayan diganti, dapur **tetap bekerja**. Resep berubah, pelayan **tidak perlu** belajar ulang.

Restoran	KedaiKita
Pelayan	Lapisan **tampilan**: `AplikasiKasir`
Kepala dapur	Lapisan **layanan**: `LayananKasir`
Juru masak dan resepnya	Lapisan **model**: `Menu`, `Pesanan`, dan lainnya
Gudang dan buku catatan	Lapisan **penyimpanan**: `RepositoriMenuCSV`

Empat Lapisan KedaiKita

Lapisan	Kelas	Tugasnya
Tampilan	`AplikasiKasir`	bertanya ke kasir dan menampilkan hasil
Layanan	`LayananKasir`	mengatur alur kerja kasir
Model	`Kedai`, `Menu`, `Pesanan`, `Pembayaran`, `Diskon`, `StrukPesanan`, `LaporanPenjualan`	menyimpan aturan bisnis KedaiKita
Penyimpanan	`RepositoriMenuCSV`, `PencatatTransaksiCSV`	membaca dan menulis *file* CSV di folder `data/`

Lapisan atas boleh memakai lapisan di bawahnya, tetapi tidak sebaliknya. Model hanya **memberi tahu** pencatat lewat *Observer*.

Tugas Setiap Lapisan

Lapisan	Kelas	Tugasnya	Tidak boleh
Tampilan (*presentation*)	`AplikasiKasir`	menampilkan menu pilihan, membaca ketikan kasir, mencetak hasil	menghitung pajak atau diskon sendiri
Layanan (*service*)	`LayananKasir`	alur kerja: daftar pelanggan, buat pesanan, tambah item, bayar, laporan	membaca *keyboard* atau mencetak menu pilihan
Model (*domain*)	`Kedai`, `Menu`, `Pesanan`, `Diskon`, dan lainnya	menyimpan data dan menjaga aturan bisnis: harga > 0, stok cukup, pajak 10%	tahu bahwa ada *keyboard* atau *file* CSV
Penyimpanan (*persistence*)	`RepositoriMenuCSV`, `PencatatTransaksiCSV`	membaca dan menulis *file*	memutuskan aturan bisnis

Manfaatnya: tampilan bisa diganti (misalnya versi web), mudah diuji, mudah dicari, dan mudah dibagi dalam tim.

Package di Python dan Java

Aspek	Python	Java
Penanda *package*	folder berisi *file* `__init__.py`	baris `package kedaikita;` di setiap *file*, di folder bernama sama
Satu *file* berisi	satu modul (boleh beberapa kelas)	satu kelas `public`
Nama lengkap kelas	`kedaikita.menu.Menu`	`kedaikita.Menu`
Cara memakai	`from kedaikita.menu import Menu`	`import kedaikita.Menu;` (sesama *package*: tanpa `import`)
Menjalankan	`python main.py --demo`	skrip `jalankan`, isinya `javac -d out ...` lalu `java -cp out kedaikita.Main`

Modul adalah satu *file* kode. ***Package*** adalah folder yang mengelompokkan modul atau kelas.

BAGIAN 3

3

Menyimpan Data ke *File*

Agar data hari ini masih ada besok.

Persistensi: Data yang Tetap Ada

Persistensi (*persistence*)

Kemampuan aplikasi **menyimpan data agar tetap ada** setelah program ditutup.

- **Menyimpan** (*save*): mengubah objek di memori menjadi teks di *file*.
- **Memuat** (*load*): membaca teks di *file* lalu menghidupkannya kembali menjadi objek.
- KedaiKita memakai **CSV**, satu baris per menu: `Minuman;M01;Kopi Susu Gula Aren;18000;20;dingin`.

CSV Bertitik Koma dan *File* yang Pasti Ditutup

- Pemisah kolomnya **titik koma** karena isi kolom seperti alamat sering mengandung koma.
- Excel berpengaturan Indonesia umumnya memakai titik koma sebagai pemisah kolom CSV.
- Hindari titik koma dan tanda petik ganda di isi kolom agar terbaca sama di Python dan Java.
- Selalu sebut *encoding* **UTF-8** agar *file* terbaca sama di Windows, macOS, dan Linux.

Penting

`with` (Python) dan *try-with-resources* (Java) menjamin *file* **pasti ditutup** begitu bloknya selesai, baik lancar maupun gagal.

Jangan Percaya Isi *File*

Isi baris yang rusak	Pesan yang ditampilkan
`Minuman;M02;Es Teh Manis;8rb;30;dingin` (di baris 3)	`Baris 3: harga, stok, dan level pedas harus berupa angka`
`Minuman;M01;Kopi Susu Gula Aren;18000;20`	`Baris 2: harus ada 6 kolom`
`Snack;M07;Keripik Pisang;10000;5;0`	`Baris 2: jenis 'Snack' tidak dikenal`
kode `M01` muncul dua kali (di baris 2 dan 3)	`Baris 3: kode M01 sudah dipakai`
`Minuman;M01;Kopi;0;20;dingin`	`Baris 2: Harga harus lebih dari 0`

Exception baru: `FormatDataError` (Python) atau `FormatDataException` (Java), turunan `KedaiError`/`KedaiException`.

BAGIAN 4

4

Input Aman dan *Unit Test*

Aplikasi yang tahan salah ketik dan diperiksa otomatis.

Tiga Kebiasaan Menangani *Input*

Kebiasaan	Artinya	Contoh di KedaiKita
Validasi	periksa jawaban sebelum dipakai: kosong? angka? dalam batas?	jumlah porsi harus angka bulat; kode menu harus terdaftar
Tanya ulang	jika tidak valid, beri pesan yang jelas lalu tanyakan lagi	`Harap isi dengan angka bulat, misalnya 2 atau 50000.`
Tangani akhir *input*	*input* bisa habis (EOF), misalnya pengguna menekan Ctrl+D atau Ctrl+Z	mencetak `Input berakhir.`, menyimpan stok, lalu pamit

Tampilan memeriksa **bentuk** jawaban; model menjaga **aturan bisnis** seperti stok cukup dan harga > 0.

Unit Test: Membiarkan Komputer Memeriksa

Unit test

Kode kecil yang memeriksa **satu bagian kecil** program (satu *method* atau satu kelas) secara **otomatis**. Pemeriksaannya disebut ***assertion***: “nilai ini seharusnya sama dengan itu”.

- Tiga langkah setiap *test*: **siapkan** data, **jalankan** *method*, **periksa** hasilnya.
- Uji tiga jenis kasus: **normal**, **gagal**, dan **batas** (*boundary*).
- Aturan “minimal Rp100.000” harus diuji dengan belanja **tepat** Rp100.000.

Unit Test KedaiKita di Python dan Java

Siapkan 2 Kopi Susu Gula Aren (Rp18.000) dan 1 Nasi Goreng Kampung (Rp25.000), lalu periksa hasilnya.

Python python/studi-kasus/tests/test_kedaikita.py

```python
def setUp(self):
    # ... (dilewati)
    self.pelanggan = Pelanggan("Uji Coba", "080000000000")
    self.pesanan = Pesanan(self.pelanggan)
    self.pesanan.tambah_item(self.kopi, 2)
    self.pesanan.tambah_item(self.nasi, 1)

def test_subtotal(self):
    self.assertEqual(self.pesanan.hitung_subtotal(), 61000)

def test_total_sudah_termasuk_pajak_10_persen(self):
    self.assertEqual(self.pesanan.hitung_total(), 67100)
```

Java java/studi-kasus/src/kedaikita/UjiKedaiKita.java

```java
private static void siapkanPesanan() throws KedaiException {
    // ... (dilewati)
    pelanggan = new Pelanggan("Uji Coba", "080000000000");
    pesanan = new Pesanan(pelanggan);
    pesanan.tambahItem(kopi, 2);
    pesanan.tambahItem(nasi, 1);
}

private static boolean ujiSubtotal() throws KedaiException {
    siapkanPesanan();
    return pesanan.hitungSubtotal() == 61000;
}
```



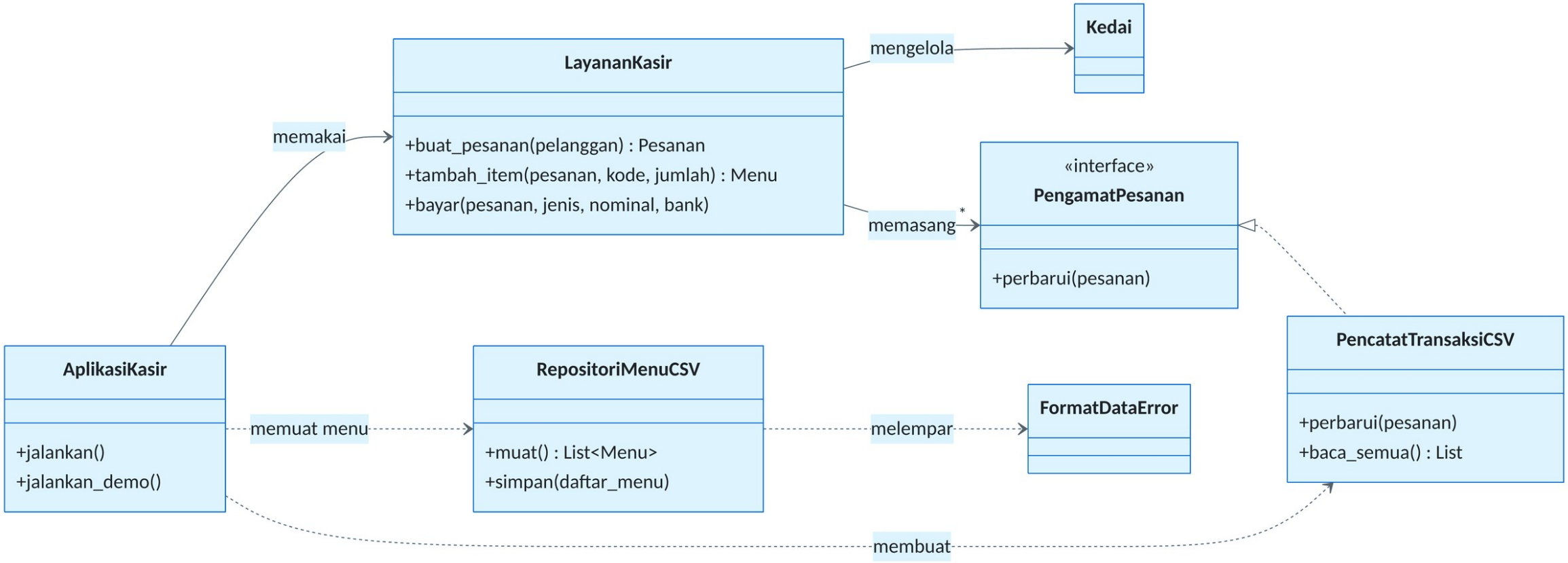
STUDI KASUS

Studi Kasus KedaiKita

Semua kelas pindah ke *package* `kedaikita`, menu dibaca dari CSV, dan ada kelas baru untuk setiap lapisan.

Kelas-Kelas Baru KedaiKita

Studi kasus KedaiKita



Nama *method* bergaya Python; di Java ditulis *camelCase*, misalnya `jalankanDemo`.

Observer Baru: Mencatat Transaksi LUNAS

Studi kasus KedaiKita

`PencatatTransaksiCSV` menambah satu baris ke `transaksi.csv` untuk setiap pesanan LUNAS. Kelas `Pesanan` tidak diubah.

Python python/studi-kasus/kedaikita/repositori.py

```python
class PencatatTransaksiCSV(PengamatPesanan):
    # ... (dilewati)
    def perbarui(self, pesanan: Pesanan):
        if pesanan.status != "LUNAS":
            return
        porsi = 0
        for item in pesanan.daftar_item:
            porsi += item.jumlah
        # ... susun baris: nomor, pelanggan, porsi, total
        with open(self._path, "a", ...) as berkas:
            # ... (dilewati)
            penulis.writerow(baris)
```

Java java/studi-kasus/src/kedaikita/PencatatTransaksiCSV.java

```java
@Override
public void perbarui(Pesanan pesanan) {
    if (!pesanan.getStatus().equals("LUNAS")) {
        return;
    }
    int porsi = 0;
    for (ItemPesanan item : pesanan.getDaftarItem()) {
        porsi += item.getJumlah();
    }
    // ... susun baris: nomor, pelanggan, porsi, total
    try {
        // ... tambahkan baris ke transaksi.csv (APPEND)
    } catch (IOException e) {
        throw new UncheckedIOException(e.getMessage(), e);
    }
}
```

Pintu Masuk: Mode Demo atau Mode Interaktif

Studi kasus KedaiKita

Satu aplikasi, dua cara menjalankan. Dengan `--demo`, aplikasi berjalan sendiri dan hasilnya selalu sama.

Python python/studi-kasus/main.py

```python
from kedaikita.aplikasi import AplikasiKasir
# ... (dilewati)
FOLDER_DATA = Path(__file__).resolve().parent / "data"

def main():
    aplikasi = AplikasiKasir(FOLDER_DATA)
    try:
        if "--demo" in sys.argv:
            aplikasi.jalankan_demo()
        else:
            aplikasi.jalankan()
    except (KedaiError, OSError) as e:
        print(f"Terjadi kesalahan: {e}")
        sys.exit(1)
```

Java java/studi-kasus/src/kedaikita/Main.java

```java
private static final Path FOLDER_DATA = Path.of("data");

public static void main(String[] args) {
    AplikasiKasir aplikasi = new AplikasiKasir(FOLDER_DATA);
    try {
        if (List.of(args).contains("--demo")) {
            aplikasi.jalankanDemo();
        } else {
            aplikasi.jalankan();
        }
    } catch (KedaiException | IOException | ... e) {
        // ... (dilewati)
        System.exit(1);
    }
}
```

Miskonsepsi yang Sering Terjadi

Anggapan keliru	Yang benar
“Arsitektur berlapis artinya *file*-nya dipisah-pisah saja.”	Intinya **arah ketergantungan**: tampilan memakai layanan, layanan memakai model, dan model tidak tahu apa-apa soal *keyboard* atau *file*.
“Validasi cukup di tampilan saja.”	Tampilan memeriksa bentuk jawaban, tetapi aturan bisnis (harga > 0, stok cukup) tetap dijaga model. Data juga bisa masuk dari *file* atau *unit test*.
“Semua *test* lulus berarti programnya bebas *bug*.”	*Test* hanya memeriksa kasus yang ditulis. Kasus batas yang tidak diuji tetap bisa menyimpan *bug*.

Kuis Cepat

Di KedaiKita, kode yang mencetak teks `Pilihan:` lalu membaca ketikan kasir seharusnya berada di lapisan ...

A. Model karena `Menu` dan `Pesanan` paling tahu datanya

B. Penyimpanan karena ketikan kasir akhirnya disimpan ke *file*

C. Layanan karena `LayananKasir` mengatur alur kerja

D. Tampilan, yaitu `AplikasiKasir`

Kuis Cepat: Jawaban

Di KedaiKita, kode yang mencetak teks `Pilihan:` lalu membaca ketikan kasir seharusnya berada di lapisan ...

A. Model karena `Menu` dan `Pesanan` paling tahu datanya

B. Penyimpanan karena ketikan kasir akhirnya disimpan ke *file*

C. Layanan karena `LayananKasir` mengatur alur kerja

✓ Tampilan, yaitu `AplikasiKasir`

Jawaban: D

Tanya jawab dengan pengguna (mencetak pertanyaan, membaca *keyboard*, memeriksa bentuk jawaban) adalah tugas lapisan tampilan. `LayananKasir` (C) mengatur alur kerja, tetapi menerima data yang sudah jadi dan tidak pernah membaca *keyboard*. Model (A) dan penyimpanan (B) sama sekali tidak boleh tahu ada *keyboard*.

Kuis Cepat

Aturan toko daring: “gratis ongkir jika belanja **minimal** Rp100.000”. Seorang pemrogram keliru menulis `subtotal > 100000` alih-alih `subtotal >= 100000`. *Test* dengan belanja berapa yang **paling mungkin** menangkap kekeliruan ini?

A. Rp50.000

B. Rp99.000

C. Tepat Rp100.000

D. Rp150.000

Kuis Cepat: Jawaban

Aturan toko daring: “gratis ongkir jika belanja **minimal** Rp100.000”. Seorang pemrogram keliru menulis `subtotal > 100000` alih-alih `subtotal >= 100000`. *Test* dengan belanja berapa yang **paling mungkin** menangkap kekeliruan ini?

A Rp50.000

B Rp99.000

✓ Tepat Rp100.000

D Rp150.000

Jawaban: C

Untuk Rp50.000, Rp99.000, dan Rp150.000, kode yang benar dan kode yang keliru memberi hasil yang **sama** sehingga *test*-nya tetap lulus. Hanya di **batas** (tepat Rp100.000) keduanya berbeda: kode yang keliru masih menagih ongkir. Inilah alasan kasus batas wajib diuji.

Ringkasan

- Aplikasi dibangun dengan alur berulang: **kebutuhan → rancangan UML → implementasi → pengujian**.

- **Arsitektur berlapis**: tampilan, layanan, model, penyimpanan. Lapisan atas memakai lapisan bawah, tidak sebaliknya.
- **Modul** adalah satu *file* kode; ***package*** mengelompokkannya dan memberi nama lengkap seperti `kedaikita.Menu`.
- **Persistensi** dengan CSV bertitik koma dan UTF-8; *file* rusak dilaporkan lengkap dengan nomor baris.
- `with` (Python) dan *try-with-resources* (Java) menjamin *file* selalu ditutup.
- ***Observer*** `PencatatTransaksiCSV` mencatat pesanan LUNAS tanpa mengubah kelas `Pesanan`.
- *Input* aman: validasi, tanya ulang, tangani EOF. ***Unit test*** menguji kasus normal, gagal, dan batas.

Langkah Berikutnya

- Jalankan contoh 01—06, mode demo, mode interaktif, dan *unit test* KedaiKita di jalur pilihanmu.
- Kerjakan latihan dan kuis 10 soal, lalu baca spesifikasi proyek akhir di `proyek-akhir.md`.
- Mulai tahap 1 proyek akhir: tulis kebutuhan dan rancangan di `RANCANGAN.md`.

Buka bahan pertemuan ini di GitHub:

- Konsep
- Praktik Python
- Praktik Java
- Jembatan Python ↔ Java
- Latihan
- Kuis 10 soal
- Proyek akhir

github.com/antonprafanto/PBO2026/tree/master/Pertemuan-15-Proyek-Terintegrasi

Pindai untuk membuka folder pertemuan ini di GitHub